

LAPORAN PRAKTIKUM PERTEMUAN 6
PEMROGRAMAN MOBILE



Dosen Pengampu:

Jefry Sunupurwa Asri, S.Kom., M.Kom.

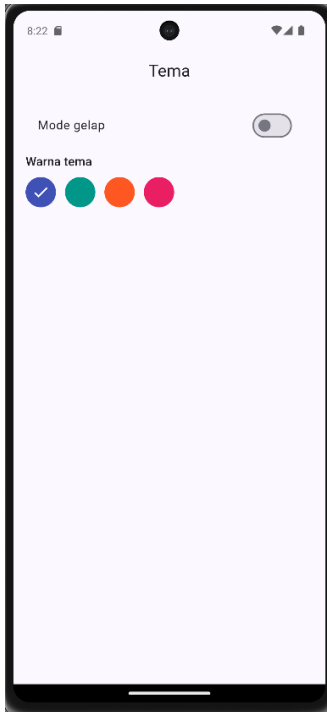
Disusun Oleh:

20240801098 – Wilson Fabian

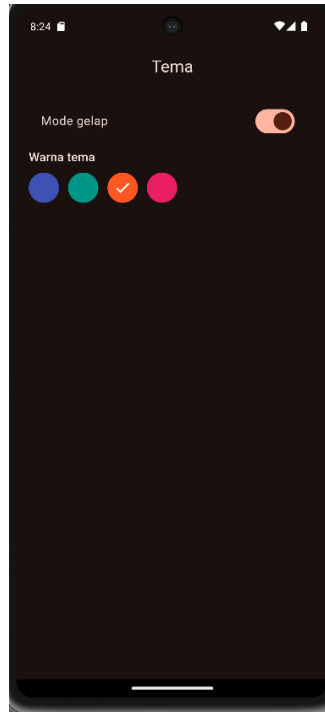
FAKULTAS ILMU KOMPUTER
UNIVERSITAS ESA UNGGUL TANGERANG

Dokumentasi Minggu 6

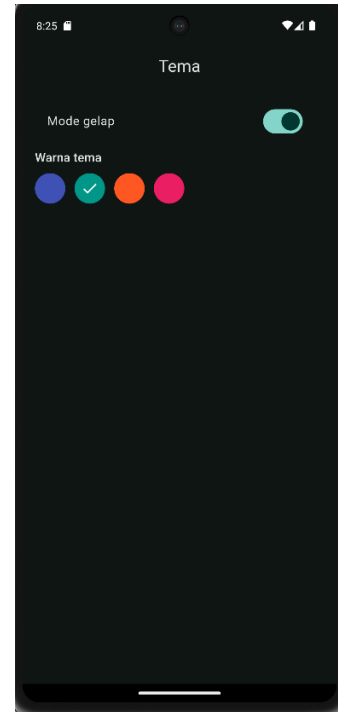
A. Tema Terang/Gelap dan Warna Tema



Mode terang, warna indigo



Mode gelap, warna deepOrange

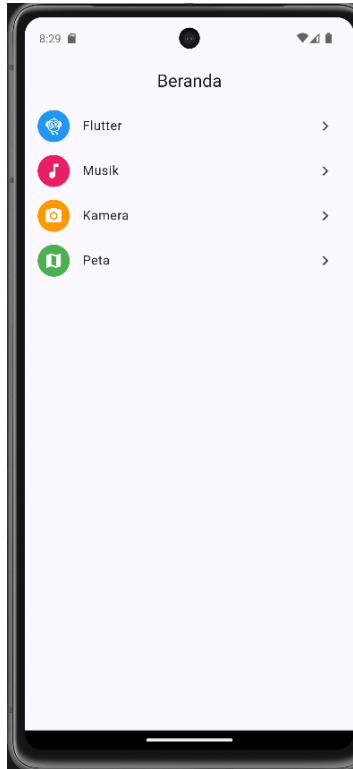


Mode gelap, warna teal

Pada bagian ini dibuat pengaturan tema aplikasi. Dua ValueNotifier global (themeMode dan seedColor) menyimpan pilihan pengguna, dan ListenableBuilder membangun ulang MaterialApp setiap kali salah satunya berubah. Fungsi buatTema() menghasilkan ThemeData Material 3 dari satu seed color, lalu dipakai pada theme (terang) dan darkTheme (gelap).

- Saklar "Mode gelap" mengubah themeMode.value antara ThemeMode.light dan ThemeMode.dark. Gambar pertama menampilkan mode terang, sedangkan gambar kedua dan ketiga mode gelap.
- Empat lingkaran warna (indigo, teal, deepOrange, pink) mengubah seedColor.value. Tanda centang menunjukkan warna yang aktif, dan seluruh palet, termasuk warna saklar, ikut berubah.
- Tidak ada warna yang ditulis manual pada widget; semuanya berasal dari tema melalui Theme.of(context), sehingga tampilan berubah langsung tanpa restart.

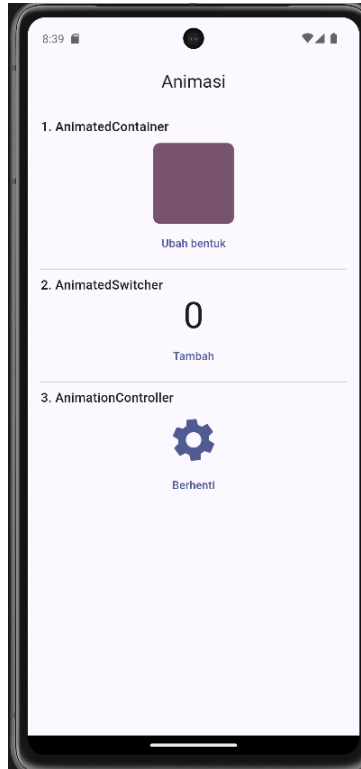
B. Named Routes, Argumen, dan Hero



Pada bagian ini ditambahkan model Item (nama, ikon, warna), data daftarItem, serta halaman BerandaTab dan DetailPage. Tangkapan layar menampilkan halaman Beranda dengan empat item: Flutter, Musik, Kamera, dan Peta.

- Setiap item berupa ListTile dengan CircleAvatar berwarna sebagai leading yang dibungkus Hero bertag 'ikon- $\{item.nama\}$ '.
- Ketika item diketuk, `Navigator.pushNamed(context, '/detail', arguments: item)` membuka rute bernama '/detail' sambil membawa objek Item.
- Rute '/' didaftarkan pada routes, sedangkan '/detail' ditangani `onGenerateRoute`. Argumen diubah menjadi Item di satu tempat lalu diteruskan ke DetailPage lewat konstruktor, sehingga tipe datanya terjaga.
- Hero dengan tag yang sama pada DetailPage membuat ikon terbang dan membesar dari daftar ke halaman detail, lalu kembali mengecil saat pengguna kembali.

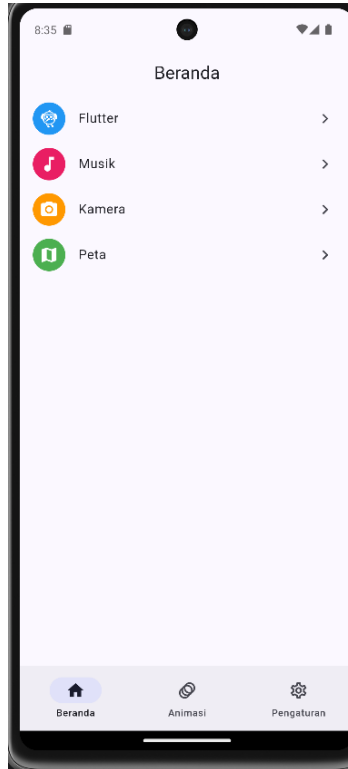
C. Animasi



Halaman AnimasiTab berisi tiga demo animasi. Karena AnimationController membutuhkan vsync, state class memakai SingleTickerProviderStateMixin.

- AnimatedContainer (animasi implisit). Tombol "Ubah bentuk" mengubah variabel `_besar` lewat `setState`, lalu lebar, warna, dan sudut kotak berubah halus selama 500 milidetik dengan kurva `Curves.easeInOut`.
- AnimatedSwitcher. Angka berganti dengan efek `ScaleTransition` setiap tombol "Tambah" ditekan. Anaknya diberi `ValueKey(_hitung)` karena AnimatedSwitcher hanya menjalankan transisi bila widget baru dianggap berbeda dari widget lama; tanpa key yang berbeda, Flutter menganggapnya widget yang sama sehingga tidak ada animasi.
- AnimationController (animasi eksplisit). Ikon roda gigi berputar terus dengan `RotationTransition` (satu putaran per 2 detik). Tombol di bawahnya berganti antara "Berhenti" dan "Putar" melalui `stop()` dan `repeat()`, dan controller dilepas pada `dispose()`.

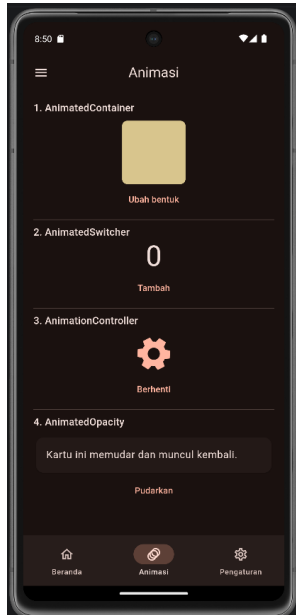
D. Navigasi Tab Bawah (Menyatukan Semuanya)



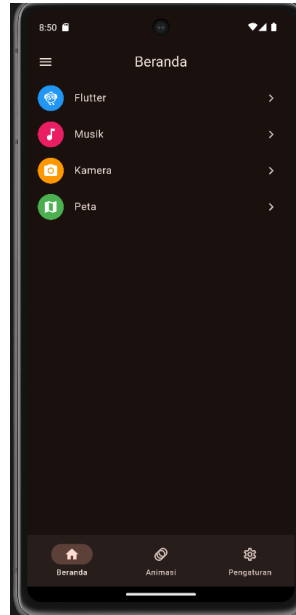
ShellPage menyatukan tiga halaman (Beranda, Animasi, dan Pengaturan) dalam satu NavigationBar di bagian bawah layar.

- IndexedStack menampilkan satu tab sesuai nilai `_index` sambil menjaga state setiap tab tetap hidup saat pengguna berpindah tab.
- NavigationBar memiliki tiga `NavigationDestination` (ikon dan label). Fungsi `onDestinationSelected` mengubah `_index` lewat `setState`, dan judul AppBar ikut berganti sesuai tab yang aktif.
- Rute `'/'` kini mengarah ke ShellPage. Pada tangkapan layar tab Beranda sedang aktif; tab Animasi menjalankan demo, dan tab Pengaturan mengganti tema seluruh aplikasi termasuk halaman detail.

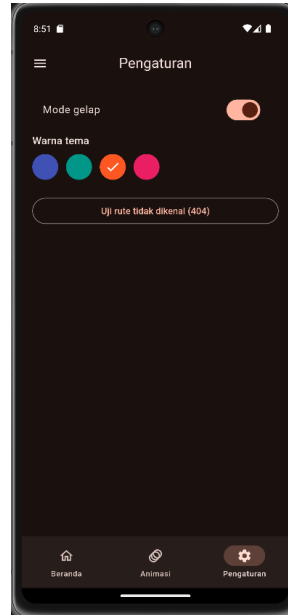
Latihan Mandiri



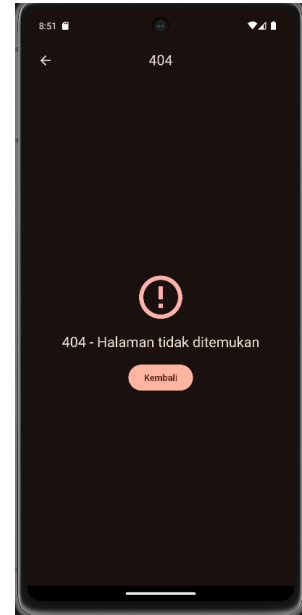
*Animasi:
AnimatedOpacity*



Beranda dengan Drawer



Tombol uji rute 404

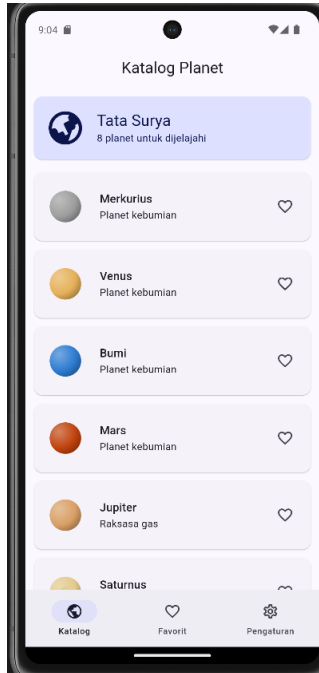


Halaman 404

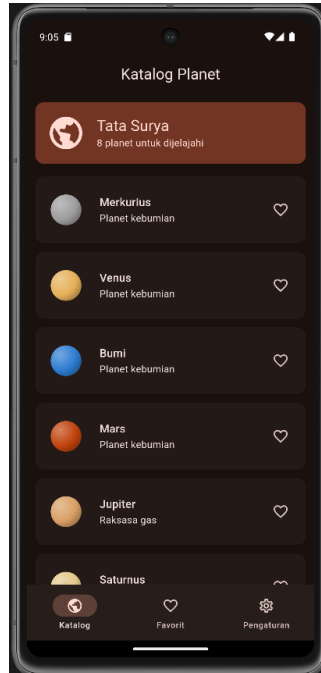
Tangkapan layar di atas menunjukkan hasil latihan mandiri (semuanya dalam mode gelap dengan warna deepOrange):

- **Drawer.** Ikon menu (tiga garis) muncul di sisi kiri AppBar pada ShellPage; ikon ini membuka Drawer berisi menu untuk berpindah tab.
- **AnimatedOpacity.** Pada AnimasiTab ditambahkan demo keempat berupa kartu "Kartu ini memudar dan muncul kembali." Tombol "Pudarkan" mengubah nilai opacity sehingga kartu memudar, lalu muncul kembali saat ditekan lagi.
- **onUnknownRoute.** Pada tab Pengaturan ditambahkan tombol "Uji rute tidak dikenal (404)" yang membuka rute yang tidak terdaftar. Rute tersebut ditangani onUnknownRoute dan menampilkan halaman "404 - Halaman tidak ditemukan" dengan ikon galat serta tombol "Kembali".

Tugas - Galeri/Katalog Interaktif (Katalog Planet)



Mode terang



Mode gelap



Halaman detail

Aplikasi bertema Katalog Planet yang berisi delapan planet Tata Surya. Aplikasi ini dibuat dengan struktur yang sama seperti latihan di modul, yaitu tema, navigasi tab, routing, dan animasi.

Halaman Utama:

- NavigationBar memiliki tiga tab: Katalog, Favorit, dan Pengaturan.
- Tab Katalog menampilkan kartu header "Tata Surya - 8 planet untuk dijelajahi", lalu daftar planet berupa kartu berisi gambar planet, nama, jenis (Planet kebumihan atau Raksasa gas), dan ikon hati untuk menandai favorit.
- Warna kartu, teks, dan latar berasal dari tema, sehingga otomatis berubah dari terang (indigo) ke gelap (deepOrange) tanpa warna yang ditulis manual.

Halaman Detail:

- Mengetuk planet membuka halaman detail melalui named route dengan membawa objek planet sebagai argumen; gambar planet memakai Hero.
- Halaman detail menampilkan gambar planet, nama, jenis, deskripsi, dan data Diameter, Jarak dari Matahari, Periode orbit, serta Jumlah bulan. Ikon hati pada AppBar dipakai untuk menandai planet sebagai favorit.
- Pilihan tema terang/gelap dan warna tema diatur pada tab Pengaturan dan disimpan dengan shared_preferences, sehingga tetap berlaku setelah aplikasi ditutup.

Pertanyaan Refleksi

1. Apa perbedaan animasi implisit dan eksplisit? Kapan memilih masing-masing?

Pada animasi implisit (`AnimatedContainer`, `AnimatedOpacity`, `AnimatedSwitcher`), kita cukup mengubah nilai properti lewat `setState` dan Flutter otomatis menganimasikan perpindahan dari nilai lama ke nilai baru. Kodenya ringkas, tetapi kontrolnya terbatas pada durasi dan kurva. Pada animasi eksplisit (`AnimationController` dengan `RotationTransition` dan sejenisnya), pengembang mengendalikan animasi secara penuh: memulai, menghentikan, mengulang, membalik arah, menyusun urutan, dan menyinkronkan beberapa animasi, namun perlu `TickerProvider` dan `dispose`. Animasi implisit dipilih untuk perubahan sederhana akibat perubahan state, seperti ukuran, warna, atau transparansi. Animasi eksplisit dipilih bila animasi harus berulang terus-menerus atau dikontrol, seperti ikon roda gigi yang berputar dan dapat dihentikan.

2. Mengapa dua Hero harus memiliki tag yang sama, dan apa yang terjadi bila tag-nya kembali muncul dua kali di halaman yang sama?

Tag adalah penanda yang dipakai Flutter untuk mengenali elemen yang sama pada dua halaman. Saat berpindah rute, Hero dengan tag yang sama di halaman asal dan tujuan dipasangkan, sehingga widget tampak terbang dan berubah ukuran serta posisi. Bila tag berbeda, tidak ada pasangan sehingga tidak ada animasi. Bila tag yang sama muncul dua kali di halaman yang sama, Flutter tidak dapat menentukan Hero mana yang dimaksud sehingga terjadi galat "multiple heroes that share the same tag within a subtree" dan animasi gagal. Karena itu tag dibuat unik untuk setiap item, misalnya 'ikon-`{item.nama}`'.

3. Apa keuntungan memakai `Theme.of(context)` dibanding menulis warna langsung pada widget?

`Theme.of(context)` membuat seluruh aplikasi bersumber pada satu tempat, yaitu `ThemeData`. Mengubah tema, baik mode terang/gelap maupun warna seed, otomatis memengaruhi semua widget tanpa restart dan tanpa mengedit widget satu per satu. Warna dari tema juga menjaga kontras dan konsistensi, misalnya pasangan `primary` dan `onPrimary`, serta ikut menyesuaikan saat mode gelap. Warna yang ditulis manual tidak mengikuti tema, sehingga mudah menimbulkan masalah seperti teks gelap di atas latar gelap, dan kode menjadi lebih sulit dirawat.

4. Apa perbedaan berpindah tab dengan `IndexedStack` dan membuka halaman dengan `Navigator`?

`IndexedStack` menampung semua tab dalam satu halaman dan hanya menampilkan satu sesuai index. Seluruh tab tetap dibuat sehingga state-nya, seperti posisi scroll atau hitungan, tetap hidup saat berpindah. Perpindahannya instan, tanpa riwayat, dan cocok untuk bagian utama aplikasi

yang sejajar. Navigator mengelola tumpukan rute: halaman baru ditumpuk di atas halaman sebelumnya dengan animasi transisi, dapat membawa argumen, dan ditutup dengan pop sehingga pengguna kembali ke halaman sebelumnya. Halaman yang sudah ditutup dibuang beserta state-nya. Navigator cocok untuk alur berjenjang seperti dari daftar ke detail.

5. Mengapa AnimationController wajib di-dispose?

AnimationController memakai Ticker yang terdaftar pada penjadwal frame dan memegang sumber daya selama aktif. Bila widget dihapus tanpa memanggil dispose(), ticker tetap berjalan di latar belakang, menyebabkan kebocoran memori dan memanggil State yang sudah tidak ada, sehingga muncul galat seperti "A Ticker was active when the State was disposed". Pada demo ini hal tersebut lebih penting karena controller dijalankan dengan repeat() terus-menerus. Oleh karena itu _putar.dispose() dipanggil pada method dispose() sebelum super.dispose().